



Testing Odoo

There are many ways to test an application. In Odoo, we have three kinds of tests

- Python unit tests (see [Testing Python code](#)): useful for testing model business logic
- JS unit tests (see [Testing JS code](#)): useful to test the javascript code in isolation
- Tours (see [Integration Testing](#)): tours simulate a real situation. They ensures that the python and the javascript parts properly talk to each other.

Testing Python code

Odoo provides support for testing modules using [Python's unittest library](#).

To write tests, simply define a `tests` sub-package in your module, it will be automatically inspected for test modules. Test modules should have a name starting with `test_` and should be imported from `tests/__init__.py`, e.g.

```
your_module
├── ...
├── tests
│   ├── __init__.py
│   ├── test_bar.py
│   └── test_foo.py
```

and `__init__.py` contains:

```
from . import test_foo, test_bar
```

Warning

test modules which are not imported from `tests/__init__.py` will not be run

The test runner will simply run any test case, as described in the official [unittest documentation](#), but Odoo provides a number of utilities and helpers related to testing Odoo content (modules, mainly):

without committing.

The data setup common to all methods should be done in the class method `setUpClass`, so that it is done once for all test methods. This is useful for test cases containing fast tests but with significant database setup common to all cases (complex in-db test data).

After being run, each test method cleans up the record cache and the registry cache. However, there is no cleanup of the registry models and fields. If a test modifies the registry (custom models and/or fields), it should prepare the necessary cleanup (`self.registry.reset_changes()`).

browse_ref(*xid*) [\[source\]](#)

Returns a record object for the provided **external identifier**

Parameters:

xid – fully-qualified **external identifier**, in the form `{module}.{identifier}`

Raise:

`ValueError` if not found

Returns:

BaseModel

ref(*xid*) [\[source\]](#)

Returns database ID for the provided **external identifier**, shortcut for `_xmlid_lookup`

Parameters:

xid – fully-qualified **external identifier**, in the form `{module}.{identifier}`

Raise:

`ValueError` if not found

Returns:

registered id

class odoo.tests.SingleTransactionCase (*methodName='runTest'*) [\[source\]](#)

TestCase in which all test methods are run in the same transaction, the transaction is started with the first test method and rolled back at the end of the last.

browse_ref(*xid*) [\[source\]](#)

Returns a record object for the provided **external identifier**

ValueError if not found

Returns:

`BaseModel`

ref(*xid*) [\[source\]](#)

Returns database ID for the provided **external identifier**, shortcut for `_xmlid_lookup`

Parameters:

xid – fully-qualified **external identifier**, in the form `{module}.{identifier}`

Raise:

ValueError if not found

Returns:

registered id

class `odoo.tests.HttpCase` (*methodName='runTest'*) [\[source\]](#)

Transactional HTTP TestCase with `url_open` and Chrome headless helpers.

browser_js(*url_path*, *code*, *ready=''*, *login=None*, *timeout=60*, *cookies=None*, *error_checker=None*, *watch=False*, *success_signal='test successful'*, *debug=False*, *cpu_throttling=None*, ****kw**) [\[source\]](#)

Test JavaScript code running in the browser.

To signal success test do: `console.log()` with the expected `success_signal`. Default is "test successful" To signal test failure raise an exception or call `console.error` with a message. Test will stop when a failure occurs if `error_checker` is not defined or returns `True` for this message

Parameters:

url_path (*string*) – URL path to load the browser page on

code (*string*) – JavaScript code to be executed

ready (*string*) – JavaScript object to wait for before proceeding with the test

login (*string*) – logged in user which will execute the test. e.g. 'admin', 'demo'

timeout (*int*) [\[source\]](#) – maximum time to wait for the test to complete (in seconds). Default is 60 seconds

cookies (*dict*) [\[source\]](#) – dictionary of cookies to set before loading the page

error_checker – function to filter failures out. If provided, the function is called with the error log message, and if it returns `False` the log is ignored and the test continue If not provided, every

debug ([bool](#)) – automatically open a fullscreen Chrome window with opened devtools and a debugger breakpoint set at the start of the tour. The tour is ran with the `debug=assets` query parameter. When an error is thrown, the debugger stops on the exception.

cpu_throttling ([int](#)) – CPU throttling rate as a slowdown factor (1 is no throttle, 2 is 2x slowdown, etc)

browse_ref(*xid*) [\[source\]](#)

Returns a record object for the provided **external identifier**

Parameters:

xid – fully-qualified **external identifier**, in the form `{module}.{identifier}`

Raise:

ValueError if not found

Returns:

BaseModel

ref(*xid*) [\[source\]](#)

Returns database ID for the provided **external identifier**, shortcut for `_xmlid_lookup`

Parameters:

xid – fully-qualified **external identifier**, in the form `{module}.{identifier}`

Raise:

ValueError if not found

Returns:

registered id

odoo.tests.tagged (**tags*) [\[source\]](#)

A decorator to tag BaseCase objects.

Tags are stored in a set that can be accessed from a 'test_tags' attribute.

A tag prefixed by '-' will remove the tag e.g. to remove the 'standard' tag.

By default, all Test classes from `odoo.tests.common` have a `test_tags` attribute that defaults to 'standard' and 'at_install'.

When using class inheritance, the tags ARE inherited.

```
# coding: utf-8
from odoo.tests import HttpCase, tagged

# This test should only be executed after all modules have been installed.
@tagged('-at_install', 'post_install')
class WebsiteVisitorTests(HttpCase):
    def test_create_visitor_on_tracked_page(self):
        Page = self.env['website.page']
```

The most common situation is to use **TransactionCase** and test a property of a model in each method:

```
class TestModelA(TransactionCase):
    def test_some_action(self):
        record = self.env['model.a'].create({'field': 'value'})
        record.some_action()
        self.assertEqual(
            record.field,
            expected_field_value)

# other tests...
```

Note

Test methods must start with `test_`

class `odoo.tests.Form` (`record`: `odoo.orm.models.BaseModel`,
`view`: `None` | `int` | `str` | `odoo.orm.models.BaseModel = None`) [\[source\]](#)

Server-side form view implementation (partial)

Implements much of the “form view” manipulation flow, such that server-side tests can more properly reflect the behaviour which would be observed when manipulating the interface:

- call the relevant onchanges on “creation”;
- call the relevant onchanges on setting fields;

changes.

Regular fields can just be assigned directly to the form. In the case of **Many2one** fields, one can assign a recordset:

```
# empty recordset => creation mode
f = Form(self.env['sale.order'])
f.partner_id = a_partner
so = f.save()
```

One can also use the form as a context manager to create or edit a record. The changes are automatically saved at the end of the scope:

```
with Form(self.env['sale.order']) as f1:
    f1.partner_id = a_partner
    # f1 is saved here

# retrieve the created record
so = f1.record

# call Form on record => edition mode
with Form(so) as f2:
    f2.payment_term_id = env.ref('account.account_payment_term_15days')
    # f2 is saved here
```

For **Many2many** fields, the field itself is a **M2MProxy** and can be altered by adding or removing records:

```
with Form(user) as u:
    u.group_ids.add(env.ref('account.group_account_manager'))
    u.group_ids.remove(id=env.ref('base.group_portal').id)
```

Finally **One2many** are reified as **O2MProxy**.

Because the **One2many** only exists through its parent, it is manipulated more directly by creating “sub-forms” with the **new()** and **edit()** methods. These would normally be used as context managers since they get saved in the parent record:

```

with f3.order_line.new() as line:
    line.product_id = env.ref('product.product_product_2')
# add a computer
with f3.order_line.new() as line:
    line.product_id = env.ref('product.product_product_3')
# we actually want 5 computers
with f3.order_line.edit(1) as line:
    line.product_uom_qty = 5
# remove support
f3.order_line.remove(index=0)
# SO is saved here

```

Parameters:

record – empty or singleton recordset. An empty recordset will put the view in “creation” mode from default values, while a singleton will put it in “edit” mode and only load the view’s data.

view – the id, xmlid or actual view object to use for onchanges and view constraints. If none is provided, simply loads the default view for the model.

New in version 12.0.

save() [\[source\]](#)

Save the form (if necessary) and return the current record:

- does not save `readonly` fields;
- does not save unmodified fields (during edition) — any assignment or onchange return marks the field as modified, even if set to its current value.

When nothing must be saved, it simply returns the current record.

Raises:

AssertionError [\[source\]](#)  – if the form has any unfilled required field

property `record`

Return the record being edited by the form. This attribute is readonly and can only be accessed when the form has no pending changes.

class `odoo.tests.M2MProxy(form, field_name)` [\[source\]](#)

Proxy object for editing the value of a many2many field.

Adds `record` to the field, the record must already exist.

The addition will only be finalized when the parent record is saved.

`remove(id=None, index=None)` [\[source\]](#)

Removes a record at a certain index or with a provided id from the field.

`clear()` [\[source\]](#)

Removes all existing records in the m2m

`class odoo.tests.O2MProxy(form, field_name)` [\[source\]](#)

Proxy object for editing the value of a one2many field.

`new()` [\[source\]](#)

Returns a **Form** for a new **One2many** record, properly initialised.

The form is created from the list view if editable, or the field's form view otherwise.

Raises:

`AssertionError` [\[source\]](#) – if the field is not editable

`edit(index)` [\[source\]](#)

Returns a **Form** to edit the pre-existing **One2many** record.

The form is created from the list view if editable, or the field's form view otherwise.

Raises:

`AssertionError` [\[source\]](#) – if the field is not editable

`remove(index)` [\[source\]](#)

Removes the record at `index` from the parent form.

Raises:

`AssertionError` [\[source\]](#) – if the field is not editable

Running tests

Test selection

In Odoo, Python tests can be tagged to facilitate the test selection when running tests.

Subclasses of `odoo.tests.BaseCase` (usually through `TransactionCase` or `HttpCase`) are automatically tagged with `standard` and `at_install` by default.

Invocation

`--test-tags` can be used to select/filter tests to run on the command-line. It implies `--test-enable`, so it's not necessary to specify `--test-enable` when using `--test-tags`.

This option defaults to `+standard` meaning tests tagged `standard` (explicitly or implicitly) will be run by default when starting Odoo with `--test-enable`.

When writing tests, the `tagged()` decorator can be used on **test classes** to add or remove tags.

The decorator's arguments are tag names, as strings.

Danger

`tagged()` is a class decorator, it has no effect on functions or methods

Tags can be prefixed with the minus (-) sign, to *remove* them instead of add or select them e.g. if you don't want your test to be executed by default you can remove the `standard` tag:

```
from odoo.tests import TransactionCase, tagged

@tagged('-standard', 'nice')
class NiceTest(TransactionCase):
    ...
```

This test will not be selected by default, to run it the relevant tag will have to be selected explicitly:

```
$ odoo-bin --test-tags nice
```

```
$ odoo-bin --test-tags nice,standard
```

The config switch parameter also accepts the `+` and `-` prefixes. The `+` prefix is implied and therefore, totally optional. The `-` (minus) prefix is made to deselect tests tagged with the prefixed tags, even if they are selected by other specified tags e.g. if there are `standard` tests which are also tagged as `slow` you can run all standard tests *except* the slow ones:

```
$ odoo-bin --test-tags 'standard, -slow'
```

When you write a test that does not inherit from the **BaseCase**, this test will not have the default tags, you have to add them explicitly to have the test included in the default test suite. This is a common issue when using a simple `unittest.TestCase` as they're not going to get run:

```
import unittest
from odoo.tests import tagged

@tagged('standard', 'at_install')
class SmallTest(unittest.TestCase):
    ...
```

Besides tags you can also specify specific modules, classes or functions to test. The full syntax of the format accepted by `--test-tags` is:

```
[-][tag][/module][:class][.method]
```

So if you want to test the `stock_account` module, you can use:

```
$ odoo-bin --test-tags /stock_account
```

If you want to test a specific function with a unique name, it can be specified directly:

This is equivalent to

```
$ odoo-bin --test-tags /account:TestAccountIncomingSupplierInvoice.test_supplier_in
```

if the name of the test is unambiguous. Multiple modules, classes and functions can be specified at once separated by a `,` like with regular tags.

Special tags

- `standard` : All Odoo tests that inherit from **BaseCase** are implicitly tagged standard. `--test-tags` also defaults to `standard`.

That means untagged test will be executed by default when tests are enabled.

- `at_install` : Means that the test will be executed right after the module installation and before other modules are installed. This is a default implicit tag.
- `post_install` : Means that the test will be executed after all the modules are installed. This is what you want for `HttpCase` tests most of the time.

Note that this is *not exclusive* with `at_install`, however since you will generally not want both `post_install` is usually paired with `-at_install` when tagging a test class.

Examples

Important

Tests will be executed only in installed modules. If you're starting from a clean database, you'll need to install the modules with the `-i` switch at least once. After that it's no longer needed, unless you need to upgrade the module, in which case `-u` can be used. For simplicity, those switches are not specified in the examples below.

Run only the tests from the sale module:

```
$ odoo-bin --test-tags /sale
```

Run only the tests from stock or tagged as slow:

```
$ odoo-bin --test-tags '-standard, slow, /stock'
```

Note

`-standard` is implicit (not required), and present for clarity

Testing JS code

Testing a complex system is an important safeguard to prevent regressions and to guarantee that some basic functionality still works. Since Odoo has a non trivial codebase in Javascript, it is necessary to test it.

See the [Unit testing](#) to learn about the various aspect of the front-end testing framework, or jump directly to one of the sub-sections:

- [Hoot](#)
- [Web test helpers](#)
- [Mock server](#)

Integration Testing

Testing Python code and JS code separately is very useful, but it does not prove that the web client and the server work together. In order to do that, we can write another kind of test: tours. A tour is a mini scenario of some interesting business flow. It explains a sequence of steps that should be followed. The test runner will then create a PhantomJs browser, point it to the proper url and simulate the click and inputs, according to the scenario.

Writing a test tour

Structure

To write a test tour for `your_module`, start with creating the required files:

```
|   └─ tests
|       └─ tours
|           └─ your_tour.js
└─ tests
    └─ __init__.py
    └─ test_calling_the_tour.py
└─ __manifest__.py
```

You can then:

- update `__manifest__.py` to add `your_tour.js` in the assets.

```
'assets': {
    'web.assets_tests': [
        'your_module/static/tests/tours/your_tour.js',
    ],
},
```

- update `__init__.py` in the folder `tests` to import `test_calling_the_tour`.

See also

- [Assets Bundle](#)
- [Testing Python code](#)

Javascript

1. Setup your tour by registering it.

```
import tour from 'web_tour.tour';
tour.register('rental_product_configurator_tour', {
    url: '/web', // Here, you can specify any other starting url
}, [
    // Your sequence of steps
]);
```

2. Add any step you want.

Example

```
// First step
tour.stepUtils.showAppsMenuItem(),
// Second step
{
    trigger: '.o_app[data-menu-xmlid="your_module.maybe_your_module_menu_root"]'
    isActive: ['community'], // Optional
    run: "click",
}, {
    // Third step
},
```

Example

```
{
    trigger: '.js_product:has(strong:contains(Chair floor protection)) .js_add'
    run: "click",
},
```

Example

```
{
    isActive: ["mobile", "enterprise"],
    content: "Click on Add a product link",
    trigger: 'a:contains("Add a product")',
    tooltipPosition: "bottom",
    async run(helpers) { //Exactly the same as run: "click"
        helpers.click();
    }
},
```

Here are some possible arguments for your personalized steps:

The action can be:

- A function, asynchronous, executed with the trigger's `Tip` as context (`this`) and the action helpers as parameter.
- The name of one of the action helpers, which will be run on the trigger element:

check

Ensures that the **trigger** element is checked. This helper is intended for `<input[type=checkbox]>` elements only.

clear

Clears the value of the **trigger** element. This helper is intended for `<input>` or `<textarea>` elements only.

click

Clicks the **trigger** element, performing all the relevant intermediate events.

dblclick,

Same as `click` with two repetitions.

drag_and_drop {target}

Simulates the dragging of the **trigger** element over to the `target` .

edit {content}

`clear` the element and then `fill` the `content` .

editor {content}

Focus the **trigger** element (wysiwyg) and then `press` the `content` .

fill {content}

Focus the **trigger** element and then `press` the `content` . This helper is intended for `<input>` or `<textarea>` elements only.

hover

Performs a hover sequence on the **trigger** element.

press {content}

Performs a keyboard event sequence.

range {content}

Performs a selection event sequence on **trigger** element. Select the option by its value . This helper is intended for `<select>` elements only.

selectByIndex {index}

Same as `select` but select the option by its `index` . Note that first option has index 0.

selectByLabel {label}

Same as `select` but select the option by its `label` .

uncheck

Ensures that the **trigger** element is unchecked. This helper is intended for `<input[type=checkbox]>` elements only.

- **isActive**: Optional, Activates the step only if all conditions of `isActive` array are met. - Browser is in either **desktop** or **mobile** mode. - The tour concerns either **community** or **enterprise** edition. - The tour is run in either **auto** (runbot) or **manual** (onboarding) mode.
- **tooltipPosition**: Optional, "top" , "right" , "bottom" , or "left" . Where to position the tooltip relative to the **target** when running interactive tours.
- **content**: Optional but recommended, the content of the tooltip in interactive tours, also logged to the console so very useful to trace and debug automated tours.
- **timeout**: How long to wait until the step can run , in milliseconds, 10000 (10 seconds).

Important

The last step(s) of a tour should always return the client to a "stable" state (e.g. no ongoing editions) and ensure all side-effects (network requests) have finished running to avoid race conditions or errors during teardown.

See also

- [jQuery documentation about find](#) 

Python

To start a tour from a python test, make the class inherit from **HTTPCase** , and call `start_tour` :

```
# Optional verifications
```

Writing an onboarding tour

Structure

To write an onboarding tour for `your_module`, start with creating the required files:

```
your_module
├── ...
├── data
│   └── your_tour.xml
├── static/src/js/tours/your_tour.js
└── __manifest__.py
```

You can then update `__manifest__.py` to add `your_tour.js` in the assets and `your_tour.xml` in the data.

```
'data': [
    'data/your_tour.xml',
],
'assets': {
    'web.assets_backend': [
        'your_module/static/src/js/tours/your_tour.js',
    ],
},
```

Javascript

The javascript part is the same as for :ref: the test tour `<testing/javascript/test>`.

XML

When you have your tour in the javascript registry, you can create a record `web_tour.tour` in the xml, like that:

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
```

```
</record>
</odoo>
```

- **name** : Required, the name must be the same as the one in the javascript registry.
- **sequence** : Optional; determines the order to execute the onboarding tours. Defaults to 1000.
- **url** : Optional; the url where to start the tour. If `url` is `False`, take the url from the registry. Defaults to `"/odoo"`.
- **rainbow_man_message** : Optional; will show the message in the rainbow man effect at the completion of the tour. If `rainbow_man_message` is `False`, there is no rainbow effect. Defaults to `<b>Good job!</b> You went through all steps of this tour.`

Running onboarding tours

They can all be started in their sequence order by toggling the **Onboarding** option in the user menu. You can run specific onboarding tours by going to the **Settings** ▶ **Technical** ▶ **User Interface** ▶ **Tours** and clicking on **Onboarding** or **Testing**.

- **Onboarding**: will execute the tour in interactive mode. That means the tour will show what to do and wait for interactions from the user.
- **Testing**: will execute the tour automatically. That means the tour will be executing all the step in front of the user.

Tour recorder

You can also create tours easily with the tour recorder. To do so, click on **Record** on the onboarding tours view. When started, this tool will record all your interactions in Odoo.

The created tours are flagged in the onboarding tours view as **Custom**. These tours can also be exported to a javascript file, ready to be put in your module.

Debugging tips

Observing test tours in a browser

There are three ways with different tradeoffs:

```
watch=True
```

```
self.start_tour("/web", "your_tour_name", watch=True)
```

This will automatically open a Chrome window with the tour being run inside it.

Advantages

- always works if the tour has Python setup / surrounding code, or multiple steps
- runs entirely automatically (just select the test which launches the tour)
- transactional (*should* always be runnable multiple times)

Drawbacks

- only works locally
- only works if the test / tour can run correctly locally

debug=True

When running a tour locally via the test suite, the `debug=True` parameter can be added to the `browser_js` or `start_tour` call:

```
self.start_tour("/web", "your_tour_name", debug=True)
```

This will automatically open a fullscreen Chrome window with opened devtools and a debugger breakpoint set at the start of the tour. The tour is ran with the `debug=assets` query parameter. When an error is thrown, the debugger stops on the exception.

Advantages

- Same advantages as mode `watch=True`
- Easier to debug steps

Drawbacks

- only works locally
- only works if the test / tour can run correctly locally

Run via browser

Test tours can also be launched via the browser UI by calling

```
odoo.startTour("tour_name");
```

- can be used on production or test sites, not just local instances
- allows running in “Onboarding” mode (manual steps)

Drawbacks

- harder to use with test tours involving Python setup
- may not work multiple times depending on tour side-effects

Tip

It's possible to use this method to observe or interact with tours which require Python setup:

- add a *python* breakpoint before the relevant tour is started (`start_tour` or `browser_js` call)
- when the breakpoint is hit, open the instance in your browser
- run the tour

At this point the Python setup will be visible to the browser, and the tour will be able to run.

You may want to comment the `start_tour` or `browser_js` call if you also want the test to continue afterwards, depending on the tour's side-effects.

Screenshots and screencasts during browser_js tests

When running tests that use `HttpCase.browser_js` from the command line, the Chrome browser is used in headless mode. By default, if a test fails, a PNG screenshot is taken at the moment of the failure and written in

```
'/tmp/odoo_tests/{db_name}/screenshots/'
```

Two new command line arguments were added since Odoo 13.0 to control this behavior: `--screenshots` and `--screencasts`

Introspecting / debugging steps

When trying to fix / debug a tour, the screenshots (on failure) are not necessarily sufficient. In that case it can be useful to see what's happening at some or each step.

- A step property `break: true`, in debug mode (`debug=True`).

This adds a debugger breakpoint at the start of the step. You can then add your own wherever you need.

Advantages

- very simple
- the tour continues as soon as you resume execution

Drawbacks

- page interaction is limited as all javascript is blocked

- A step property `pause: true`, in debug mode (`debug=True`).

The tour will stop at the end of the step. This allows inspecting and interacting with the page until the developer is ready to resume by typing **`play()`** in the browser console.

Advantages

- allows interacting with the page
- no useless (for this situation) debugger UI

- A step with a `run() { debugger; }` action.

This can be added to an existing step, or can be a new dedicated step. Once the step's **trigger** is matched, the execution will stop all javascript execution.

Advantages

- simple
- the tour continues as soon as you resume execution

Drawbacks

- page interaction is limited as all javascript is blocked
- the debugger is triggered after trying to find targeted element defined in the step.

Performance Testing

Query counts

One of the ways to test performance is to measure database queries. Manually, this can be tested with the `--log-sql` CLI parameter. If you want to establish the maximum number of



```
with self.assertQueryCount(11):  
    do_something()
```

Get Help

[Contact Support](#)[Ask the Odoo Community](#)